



THM
TECHNISCHE HOCHSCHULE MITTELHESSEN

**CAMPUS
GIESSEN**

MNI

Mathematik, Naturwissenschaften
und Informatik



PE5001

Echtzeitsysteme

Tasks



Inhalte der Veranstaltung

Aus dem Modulhandbuch + Ergänzungen

Echtzeit-
Betriebssysteme

Tasks, Semaphore,
Message Queues

Exceptions und
Interrupts

Timer, I/O-Subsystem

Memory Management

Rate-Monotonic
Scheduling

Synchronisation und
Kommunikation

Analysemethoden von
Echtzeit-anwendung

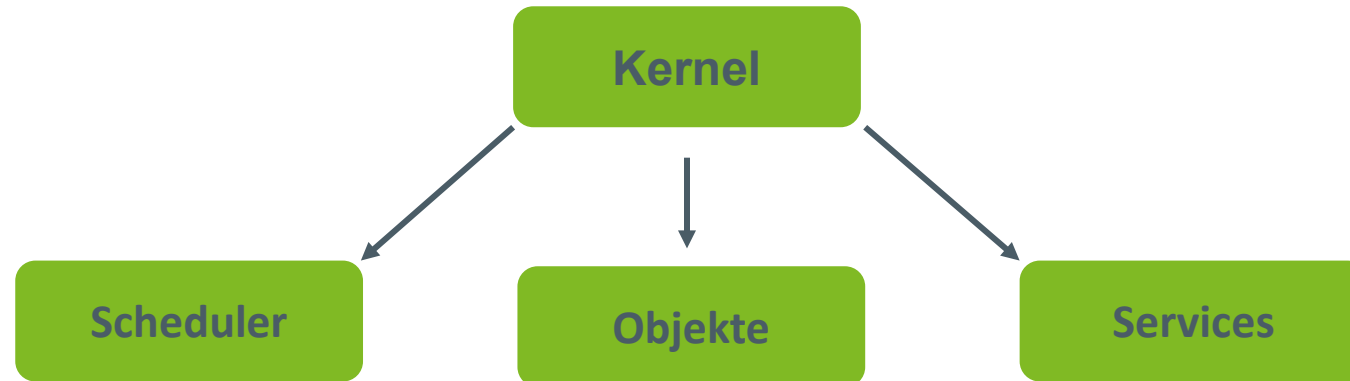


Organisatorisches

- Gruppeneinteilung abgeschlossen: Danke!
- Es gibt Teilnehmer, die nicht im Kurs angemeldet sind
- Heutiger Ablauf:
 - Task Vortrag
 - Hardware-Vergabe
 - Praktikum: Start Richtung Scheduler & Dispatching
 - Vorgaben sind in Moodle! (TCB + RTOS Details)

RTOS - Definition

- Ein *real-time operating system* (**RTOS**) ist ein Programm, welches...
 - ... Ausführungszeit zeitlich plant (Scheduler).
 - ... Systemressourcen managt.
 - ... eine Grundlage zur Entwicklung von Applikationen bildet.



RTOS

- Warum ein **RTOS**?
 - Zeitliche Restriktionen/Echtzeitanforderungen
 - Berechnungen müssen innerhalb zeitlicher grenzen erfolgen/erfolgt sein (*Deadline*)



Kaffeemaschine



Rakete



Medizinische Geräte

RTOS

soft real-time



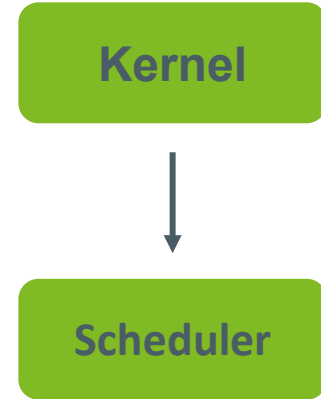
hard real-time

- Muss zeitliche Grenzen einhalten, jedoch mit einer gewissen Flexibilität
- Durch Angabe einer durchschnittlichen zeitlichen Grenze
- Eine nicht eingehaltene Deadline führt zu keinem Systemfehler. Aber Kosten können entstehen bzw. proportional zur Ausfallzeit steigen.
- Verfehlen der Zeitlichen Grenze (Deadline) beeinflusst nicht das Ergebnis. Ergebnis ist noch nützlich.

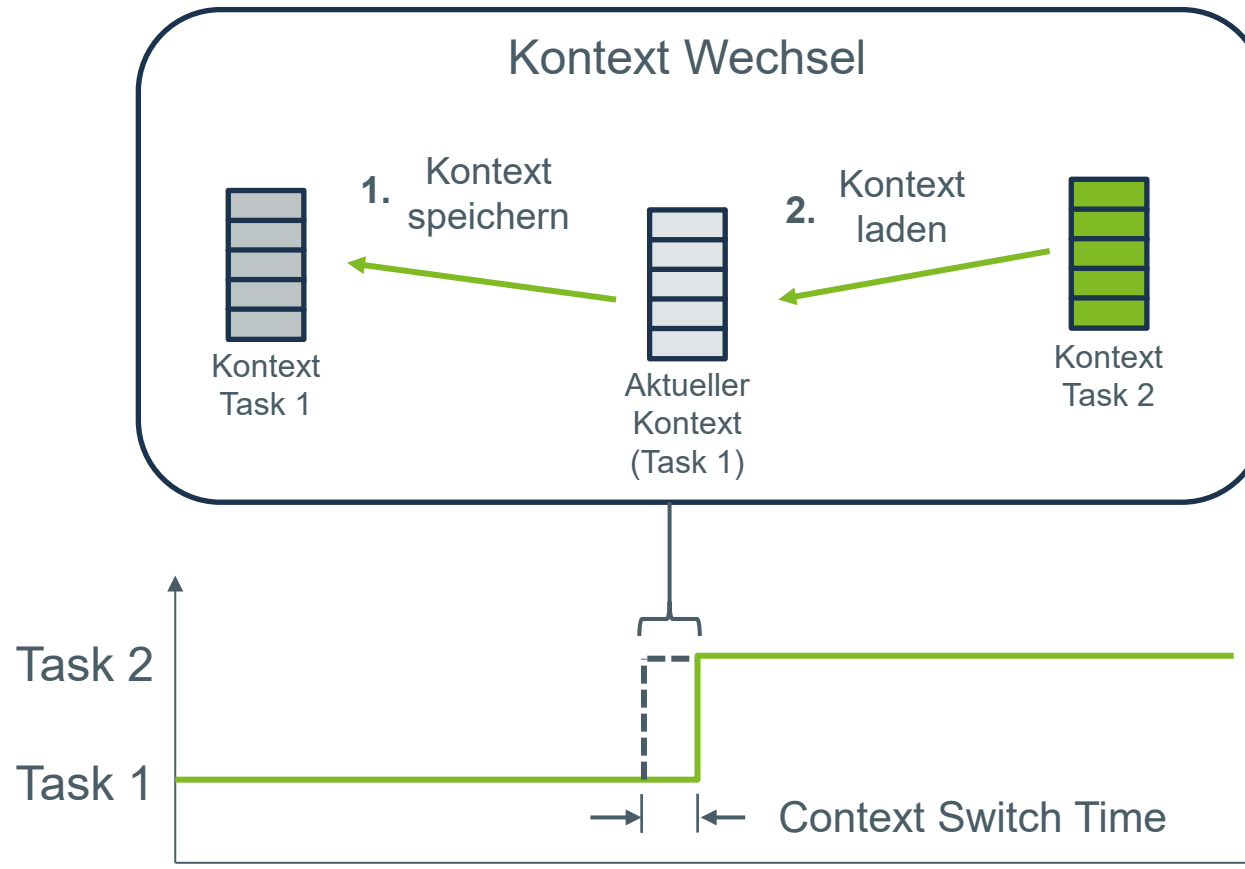
- Muss zeitliche Grenzen unbedingt einhalten – keine Flexibilität
- Kann im Worst-Case Katastrophen auslösen oder Menschenleben gefährden.
- Kann zu einem Systemfehler führen.
- Ob ein System eine harte oder weiche Echtzeitanforderung besitzt, entscheidet letztendlich die Strafe, die bei nicht-einhaltung zu erwarten ist.

Der Scheduler

- Wir wollen Tasks/Prozesse quasi-parallel ausführen
 - Das nennt man *Multitasking* (im Kontext von Single-Cores)
 - Wird erreicht durch eine verschachtelte Ausführung der Tasks
- Der Scheduler ist das Herz des RTOS.
- Er muss mit seinem Scheduling-Algorithmus dafür sorgen, dass alle Tasks bearbeitet werden und Ausführungszeit bekommen
- Das wird erreicht durch den Kontext-Wechsel (**Context Switch**). Das ist nichts anderes, als der Wechsel von einem Task in einen anderen.
- Beim **Context Switch** wird der *Zustand* des laufenden Tasks *gespeichert* und der *Zustand* des als nächsten auszuführenden Tasks *geladen*.



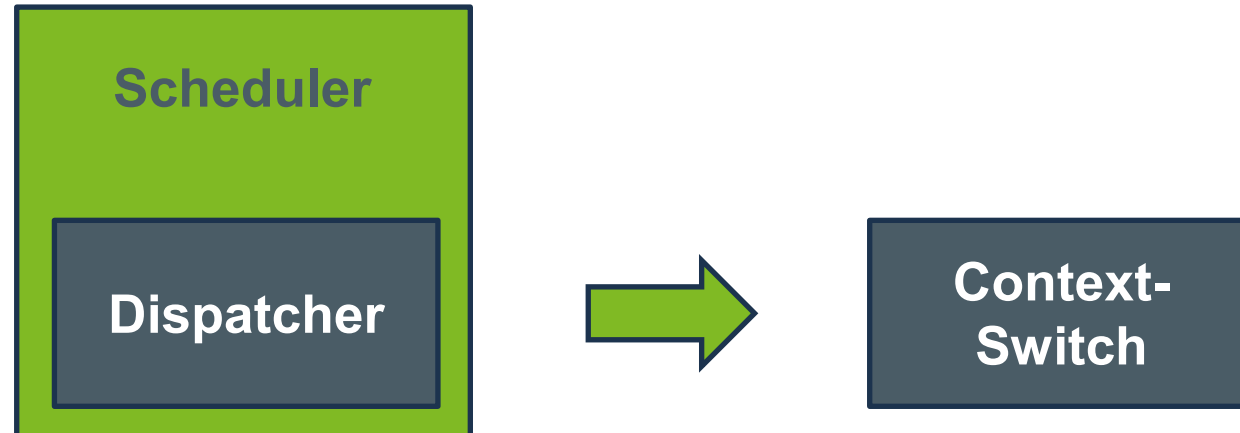
Der Kontext-Wechsel



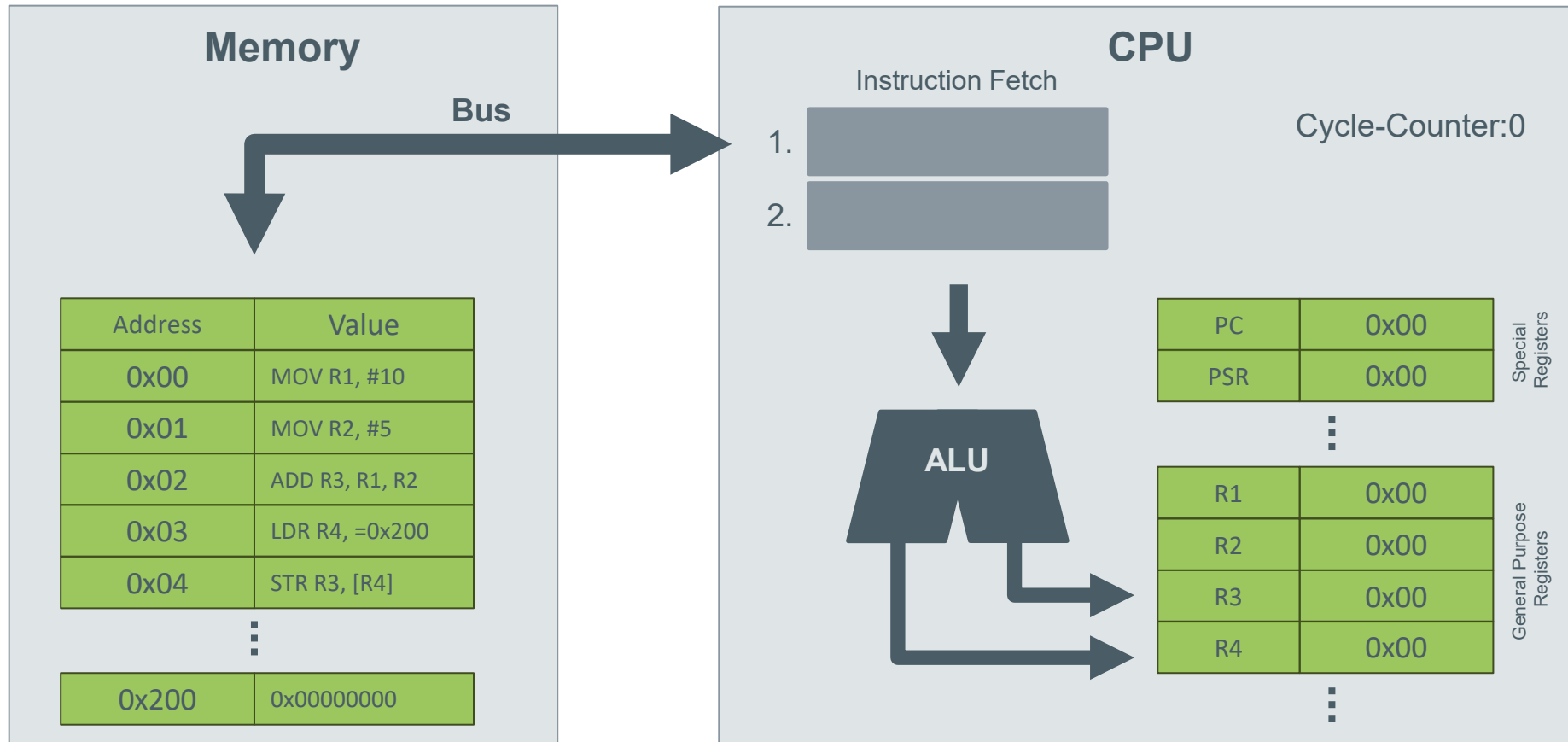
Nach Abbildung 4.3 aus Real-Time
Concepts for Embedded Systems

Der Kontext-Wechsel

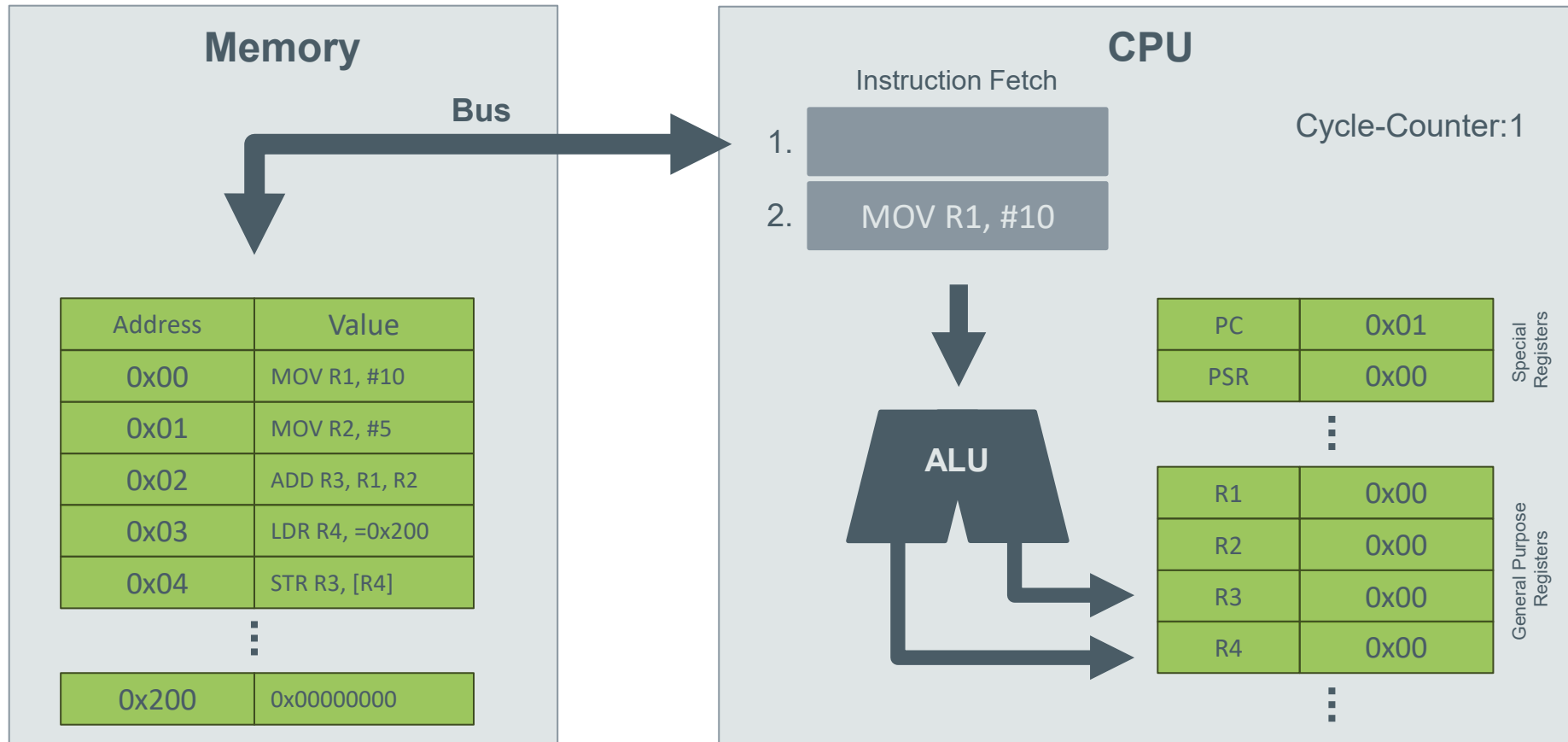
- Was ist der Zustand eines Tasks?
 - Ganz einfach. Der aktuelle Zustand des Prozessors!
 - Also die CPU-Register, Stack-Pointer etc.
- Diese Informationen speichern wir in einem **Task-Control Block (TCB)** ab.
- Der Teil unseres RTOS, welches den Context Switch durchführt, nennt man **Dispatcher**



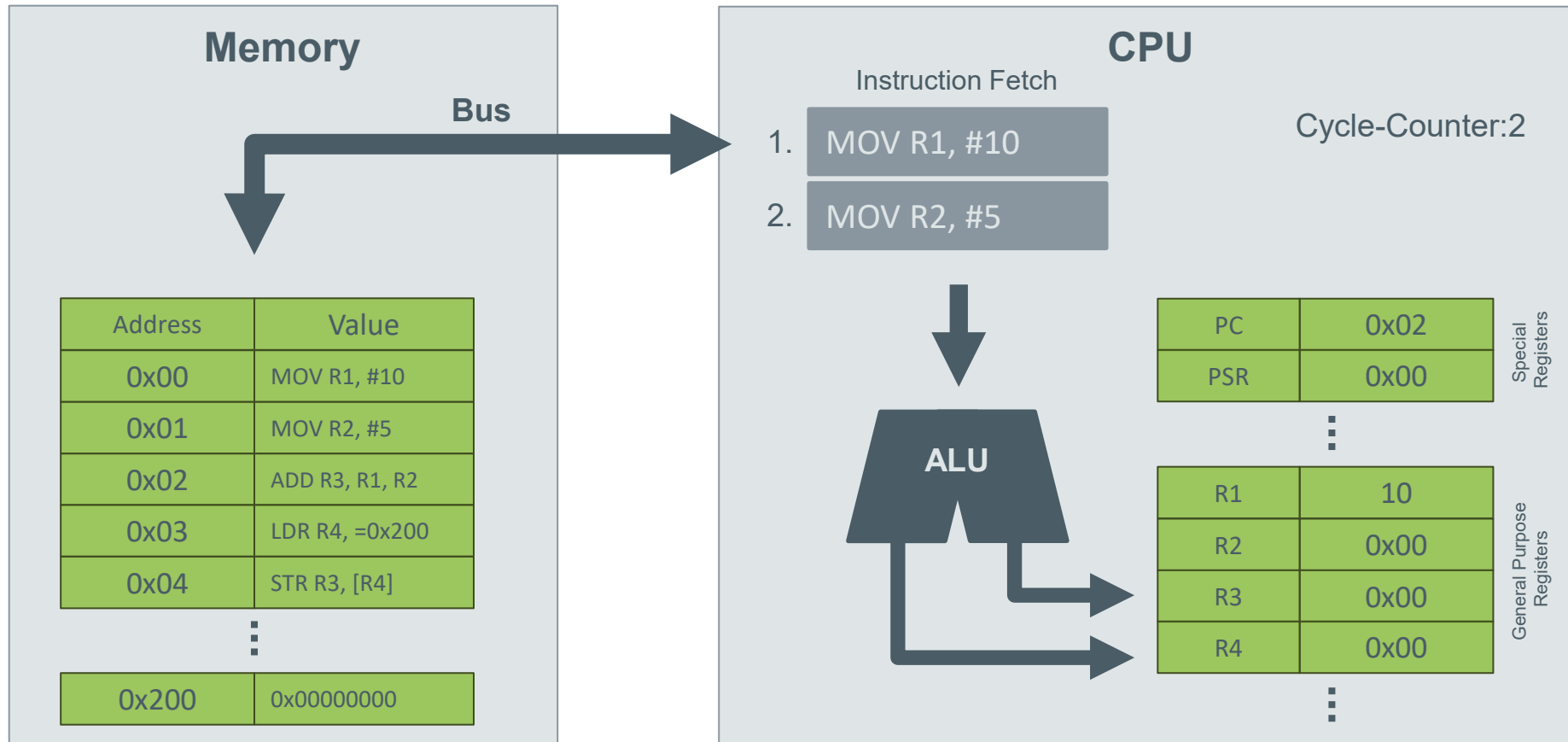
Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)



Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)

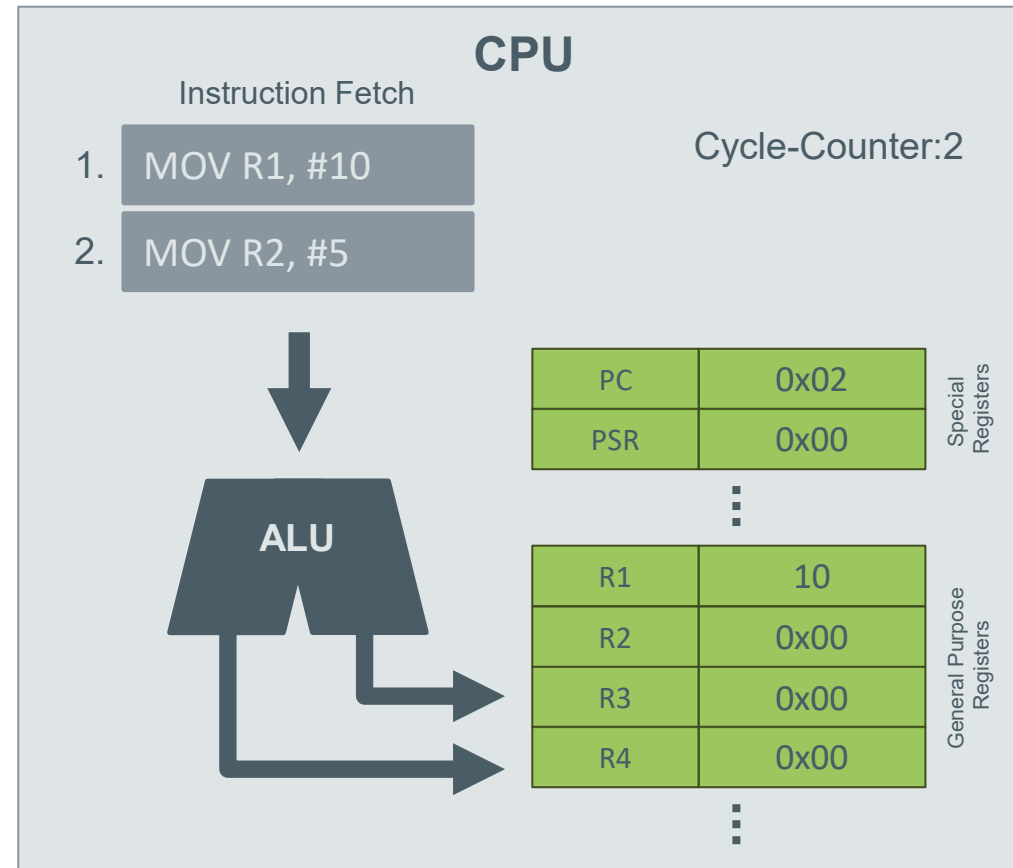


Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)

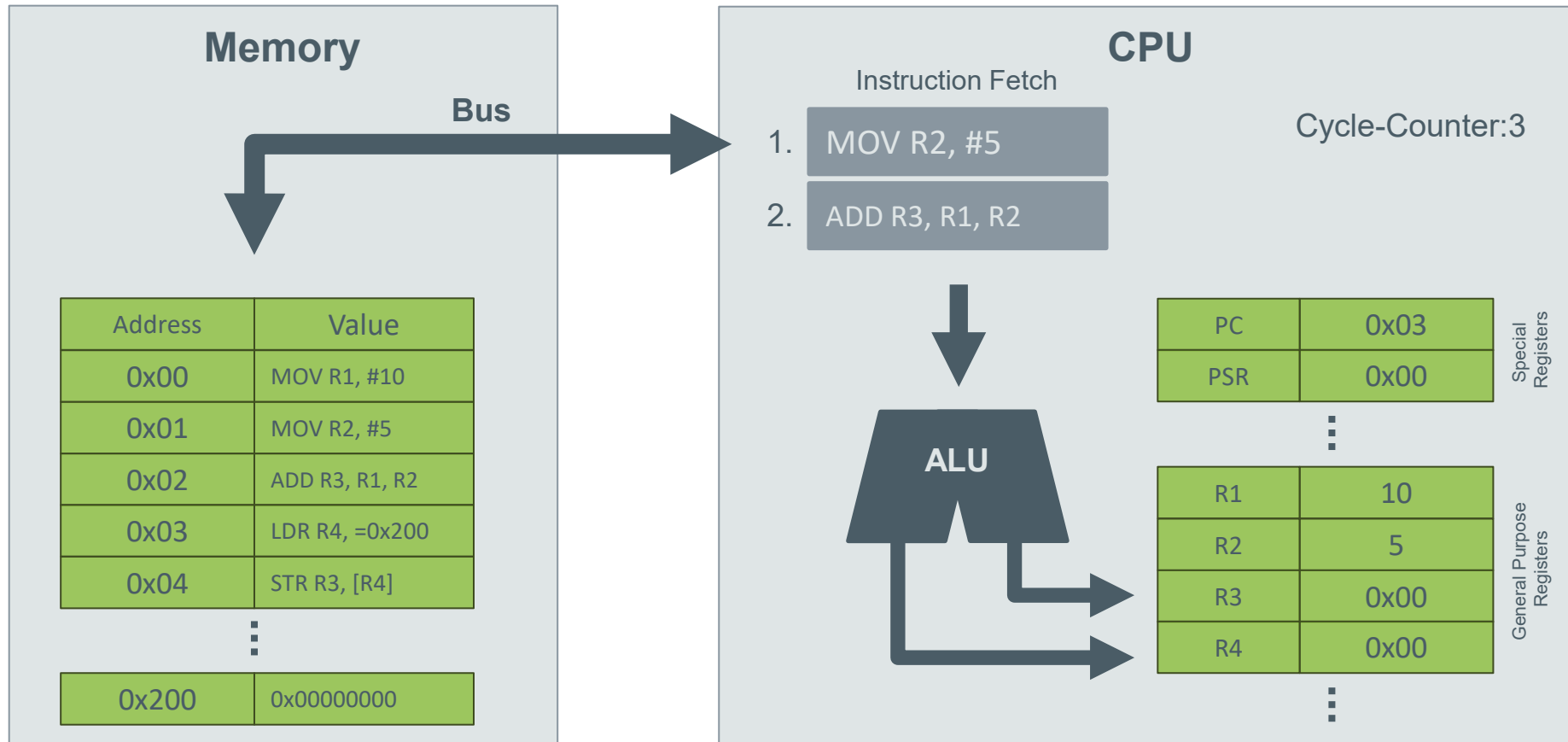


Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)

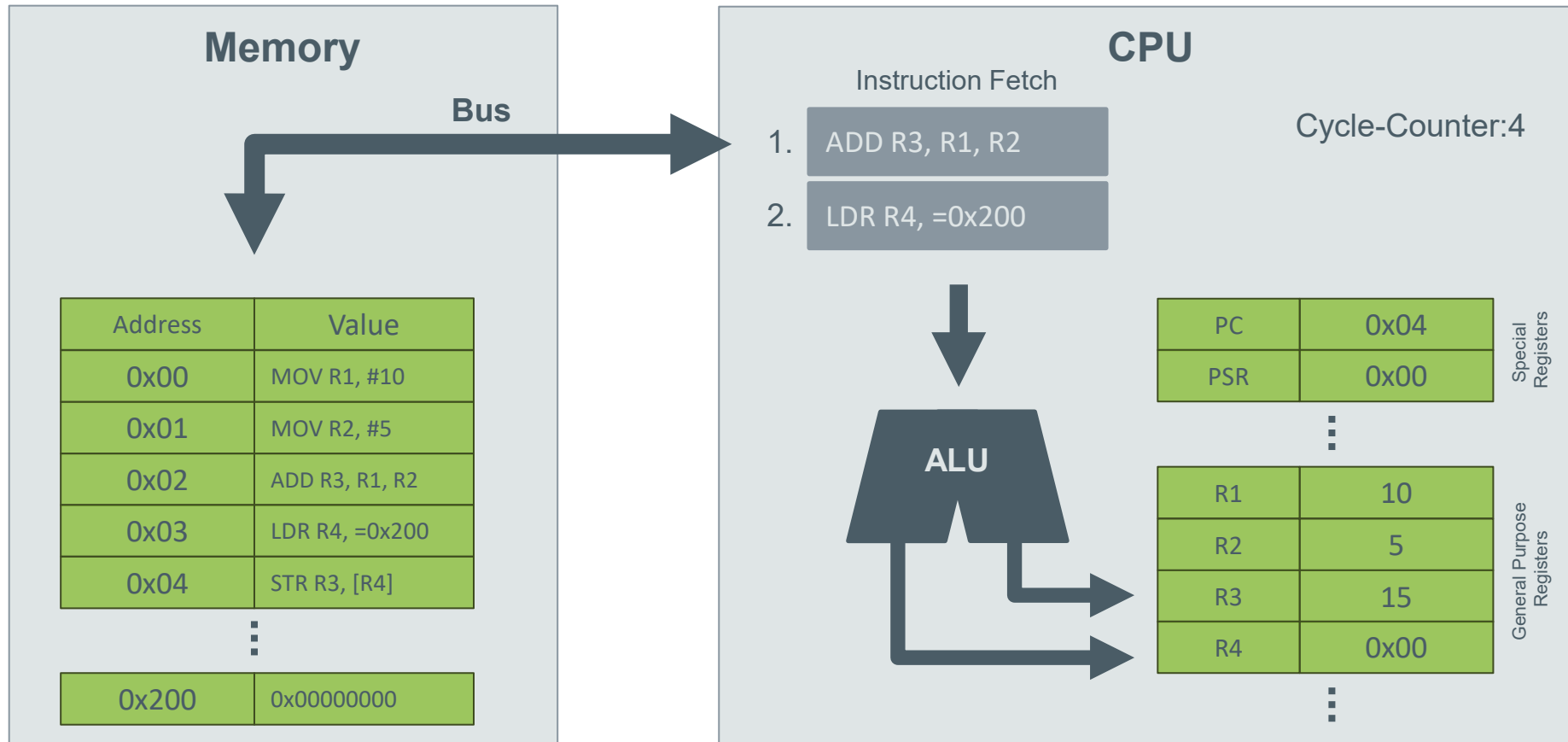
Was ist der aktuelle Zustand der CPU?



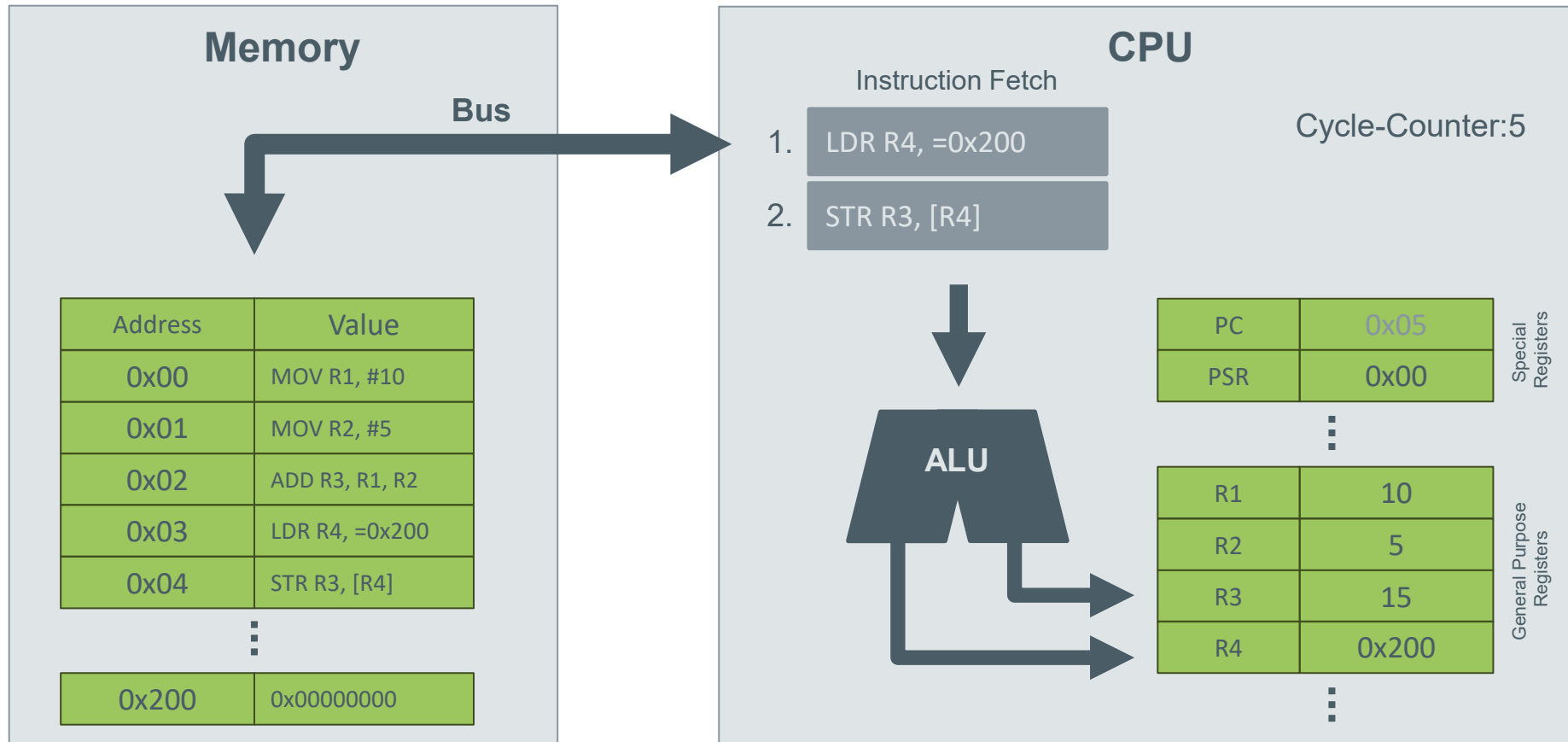
Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)



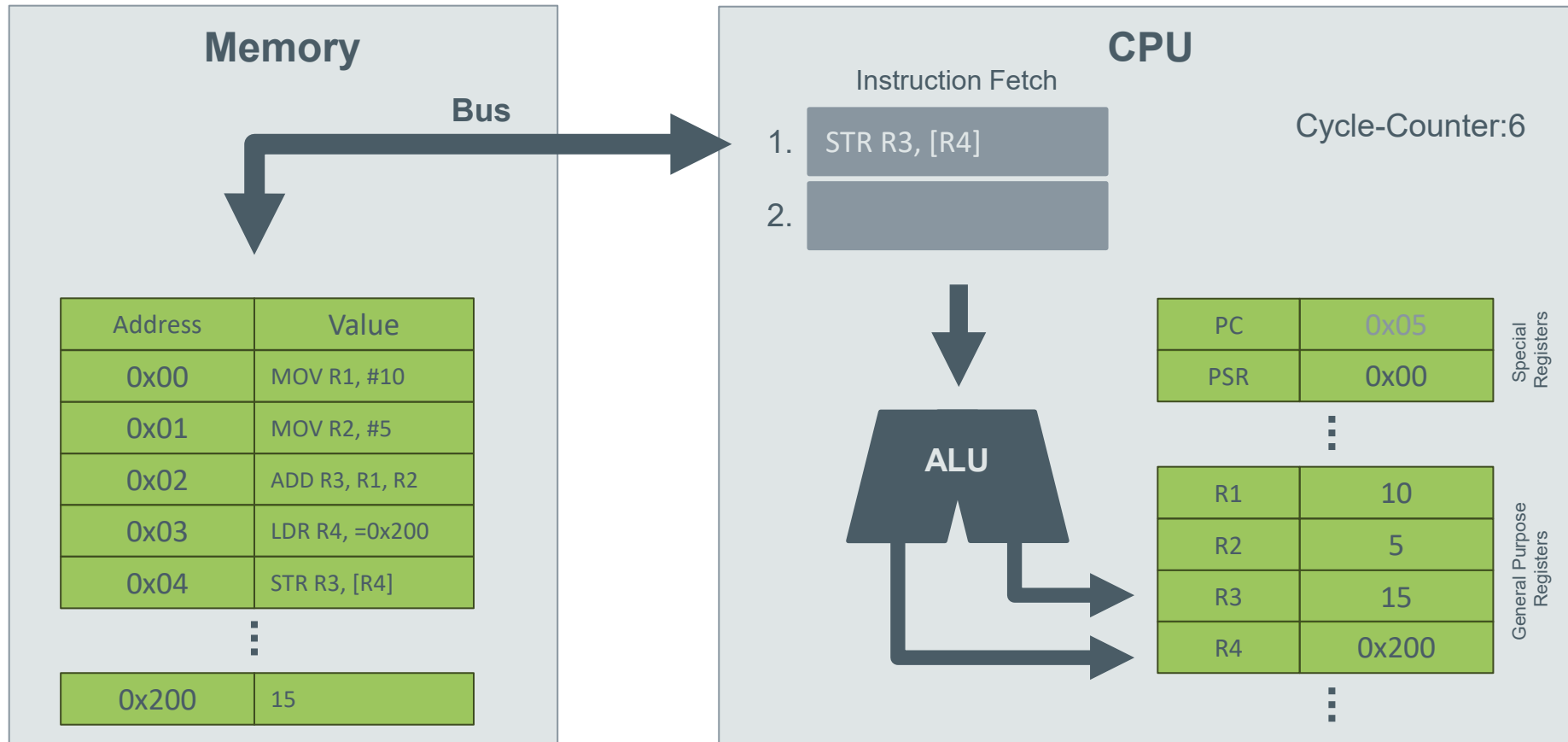
Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)



Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)

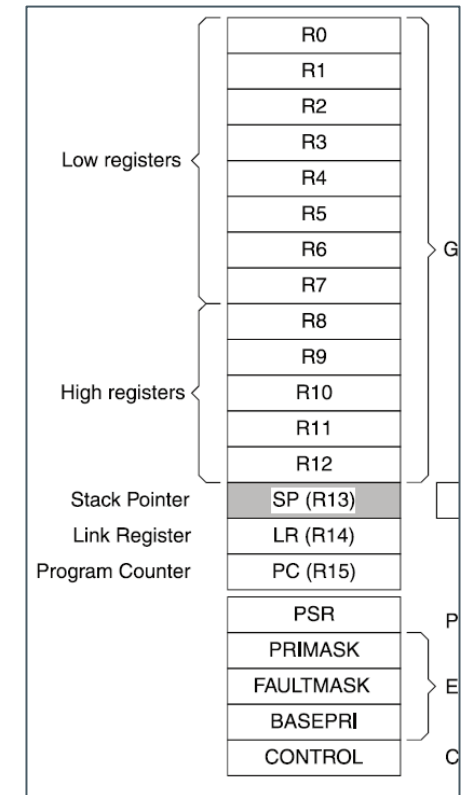
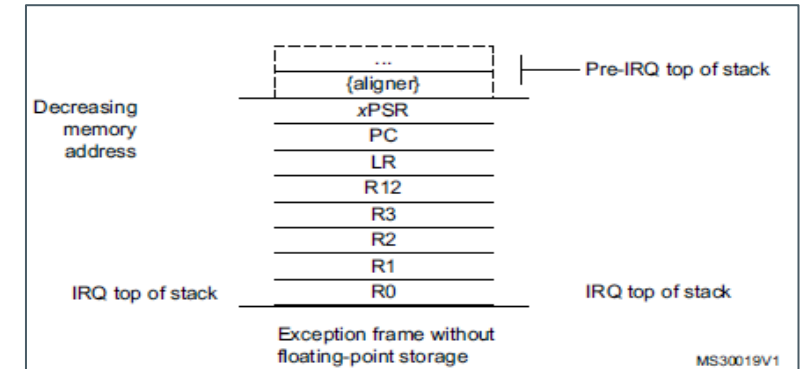
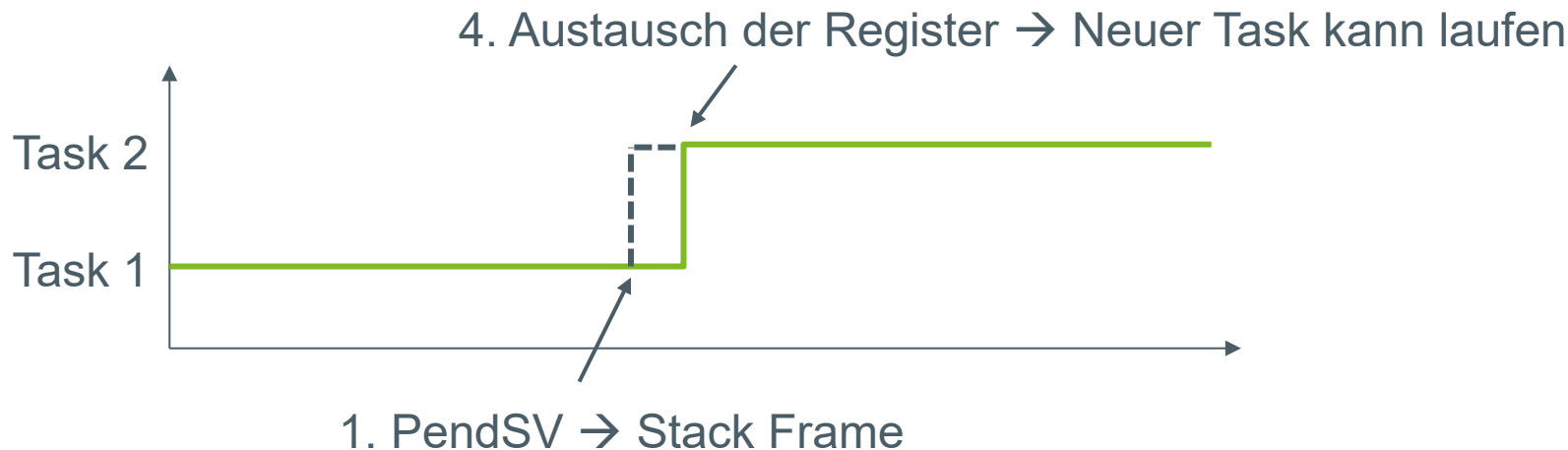


Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)



Dispatching auf dem Cortex M4

1. **PendSV** pusht unseren Stack-Frame auf den Stack (ganz automatisch).
2. Interrupts ausschalten
3. **Push R4-R11** auf den Stack (die restlichen CPU-Register, die nicht durch das Stack-Frame auf den Stack gelegt werden)
4. Laden des neuen Tasks = Stack von Task 2, also alle Register der CPU austauschen in den Zustand/Stack von Task 2
5. Interrupts einschalten
6. PendSV Routine verlassen (Stack-Frame wird vom Stack genommen)



Tasks - Definition

- Verschiedene Tasks sollen Aufgaben „parallel“ abarbeiten...
- ...und Berechnungen in Bezug auf Deadlines einhalten und rechtzeitig fertigstellen.
- „Ein Task ist ein unabhängiger „Ausführungsstrang“ von Code, welche mit anderen Tasks um Ausführungszeit des Prozessors konkurriert.“¹
- Auf einem Single-Core führen wir Tasks „quasi-parallel“ aus. Echte Parallelität erreichen wir nur durch Multi-Cores.

¹ Real-Time Concepts for Embedded Systems, p. 65

Tasks

- Aus was besteht ein Task?
 - Einem **Task Control Block (TCB)**
 - Einer ID für die Zuordnung (+ evtl. einen Namen)
 - Eine Priorität (wenn ein entsprechender Scheduler eingesetzt werden soll)
 - Einem dedizierten Task-Stack
 - Und eine Task-Routine!
 - Alles zusammen ergibt ein Task-Objekt (und das ist entsprechend auch ein Kernel-Objekt)

```

32 /// Structure for the task control block (TCB)
33 typedef struct
34 {
35     ...uint8_t u8TaskId; ... ///< Identification of a task
36     ...uint8_t u8TaskPrio; ... ///< priority of the task
37     ...TCB_eTaskStates_t eTaskState; ... ///< the tasks state
38
39     ...uint32_t au32TaskStack[TCB_TASK_STACK_SIZE]; ... ///< Tasks's
40     ...uint32_t u32TaskSP; ... ///< To store
41 } TCB_sctTCB_t;
  
```

0x20017ef4 : 0x20017EF4 <Hex> X + New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
20017EF0	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017F00	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017F10	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017F20	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017F30	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017F40	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017F50	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017F60	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017F70	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017F80	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017F90	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017FA0	AAAAAAAA	AAAAAAAA	AAAAAAAA	AAAAAAAA
20017FB0	04000000	05000000	06000000	07000000

```

/// Task handler for task 2
void Task2_vHandler(void)
{
    ...// Init
    ...for (;;)
    ...{
    ...    ...// Add work
    ...    HAL_GPIO_TogglePin(LED2_GPIO_Port, LED2_Pin);

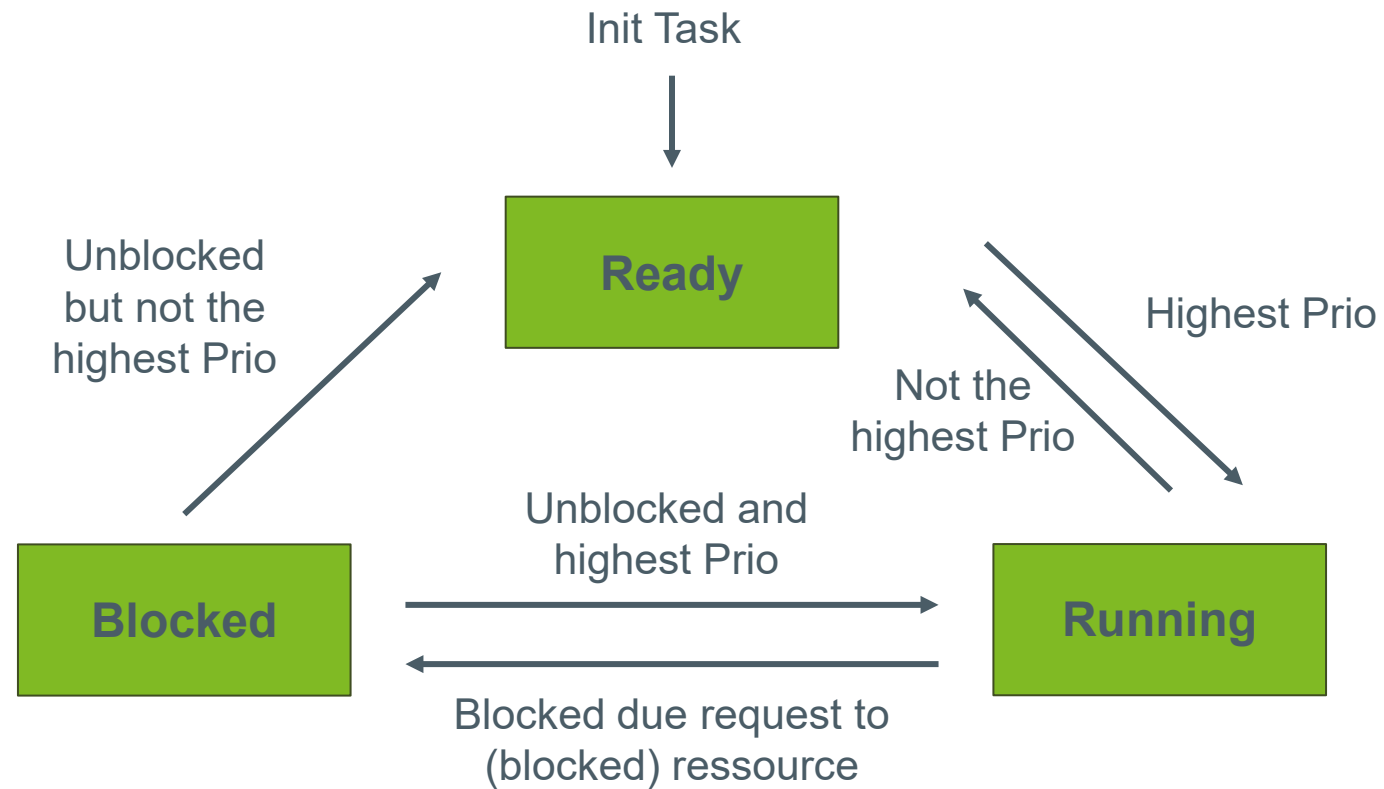
    ...    ...// Add delay which suspends the task from its work.
    ...    DOS_vTaskDelayMsBlocking(100u);
    ...}
}
  
```

Tasks

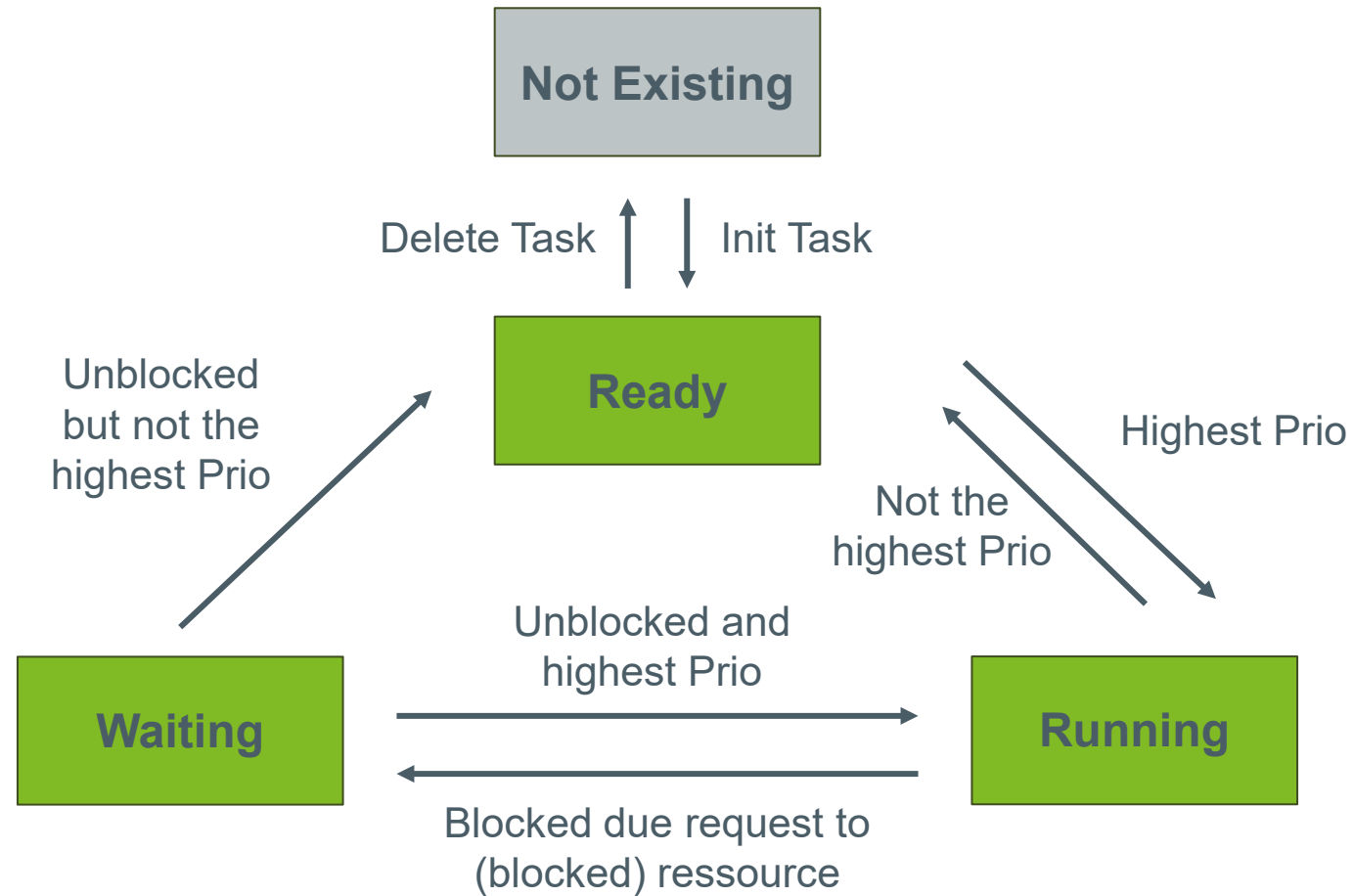
- Tasks werden vom Nutzer erstellt. Aber es gibt Ausnahmen...
 - ... was würde denn passieren, wenn wir keinen vom Nutzer erstellten Task ausführen können?
 - ... der Scheduler braucht eine Fallback-Alternative. Dafür ist der IDLE-Task zuständig.
- Das Betriebssystem erstellt den IDLE-Task selbstständig.
- Gibt es Task-Prioritäten, bekommt der IDLE-Task immer die niedrigste Priorität im System.
- Der IDLE-Task führt *normalerweise* keine Arbeit aus.
- Es können oft aber Hooks verwendet werden, um benutzerspezifischen Code auszuführen. Was könnte das sein?
 - Sleep- oder Low-Power Modes, die ausgeführt werden um Energie zu sparen.

Tasks - Zustände

- Tasks benötigen Zustände. Diese werden vom Scheduler verwendet.



Tasks - Zustände



```

graph TD
    WAITING((TASK WAITING))
    DORMANT((TASK DORMANT))
    READY((TASK READY))
    RUNNING((TASK RUNNING))
    ISR_RUNNING((ISR RUNNING))

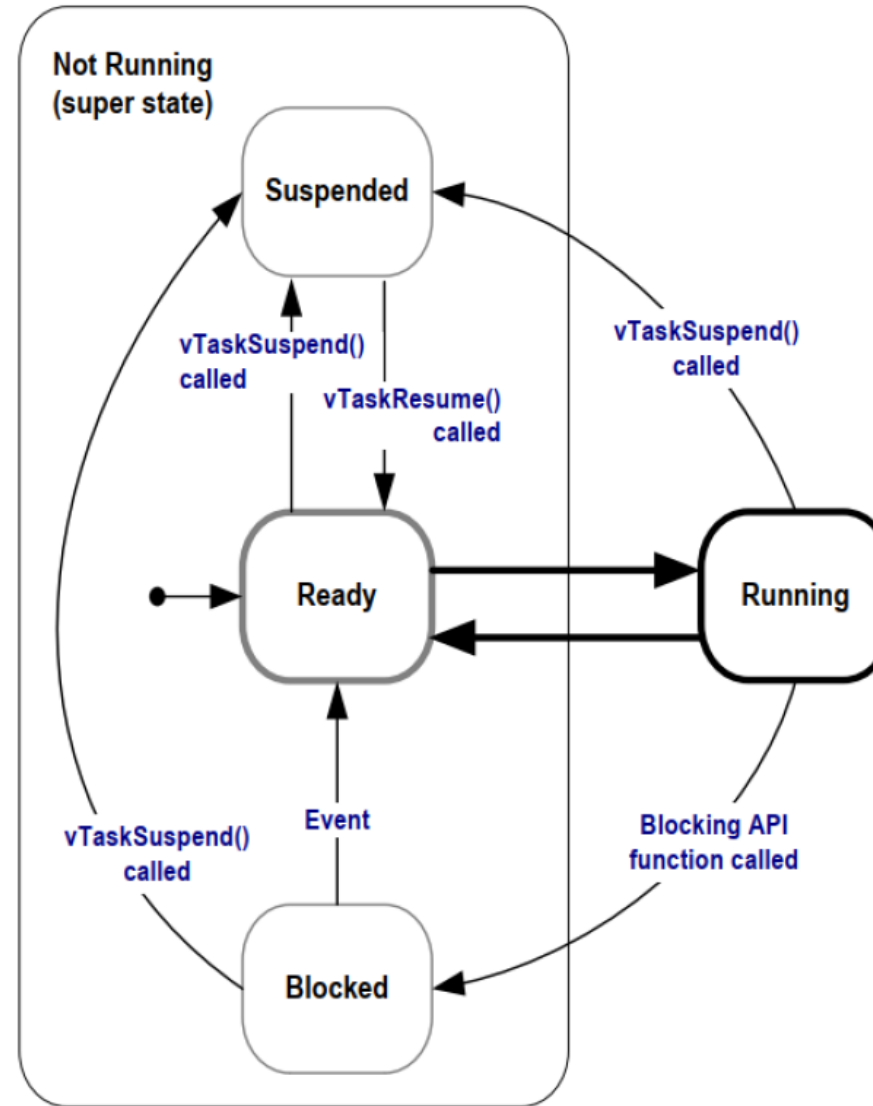
    WAITING -- "OSTaskDel()" --> DORMANT
    WAITING -- "OSFlagPost(), OSMboxPost(), OSFlagPend(), OSMboxPend(), OSMutexPost(), OSQPost(), OSQPostFront(), OSQPostOpy(), OSSemPost(), OSTaskResume(), OSTimeDlyResume(), OSTimeTick()" --> READY
    WAITING -- "OSFlagPend(), OSMboxPend(), OSMutexPend(), OSQPend(), OSSemPend(), OSTaskSuspend(), OSTimeDly(), OSTimeDlyHMSM()" --> RUNNING
    DORMANT -- "OSTaskCreate(), OSTaskCreateExt()" --> READY
    DORMANT -- "OSTaskDel()" --> DORMANT
    READY -- "OSStart(), OSIntExit(), OS_TASK_SW()" --> RUNNING
    READY -- "OSTaskDel()" --> DORMANT
    READY -- "Task is Preempted" --> RUNNING
    RUNNING -- "Interrupt" --> ISR_RUNNING
    RUNNING -- "OSIntExit()" --> READY
    RUNNING -- "OSTaskDel()" --> DORMANT
  
```

The diagram illustrates the state transitions for tasks in an operating system. The states are represented by circles: TASK WAITING, TASK DORMANT, TASK READY, TASK RUNNING, and ISR RUNNING. Transitions between these states are labeled with specific OS functions or events.

- TASK WAITING** can transition to **TASK DORMANT** via `OSTaskDel()`.
- TASK WAITING** can transition to **TASK READY** via a list of functions: `OSFlagPost()`, `OSMboxPost()`, `OSFlagPend()`, `OSMboxPend()`, `OSMutexPost()`, `OSQPost()`, `OSQPostFront()`, `OSQPostOpy()`, `OSSemPost()`, `OSTaskResume()`, `OSTimeDlyResume()`, and `OSTimeTick()`.
- TASK WAITING** can transition to **TASK RUNNING** via a list of functions: `OSFlagPend()`, `OSMboxPend()`, `OSMutexPend()`, `OSQPend()`, `OSSemPend()`, `OSTaskSuspend()`, `OSTimeDly()`, and `OSTimeDlyHMSM()`.
- TASK DORMANT** can transition to **TASK READY** via `OSTaskCreate()` and `OSTaskCreateExt()`.
- TASK DORMANT** can transition to **TASK DORMANT** via `OSTaskDel()`.
- TASK READY** can transition to **TASK RUNNING** via `OSStart()`, `OSIntExit()`, and `OS_TASK_SW()`.
- TASK READY** can transition to **TASK DORMANT** via `OSTaskDel()`.
- TASK READY** can transition to **TASK RUNNING** via `Task is Preempted`.
- TASK RUNNING** can transition to **ISR RUNNING** via `Interrupt`.
- TASK RUNNING** can transition to **TASK READY** via `OSIntExit()`.
- TASK RUNNING** can transition to **TASK DORMANT** via `OSTaskDel()`.

<https://micrium.atlassian.net/wiki/spaces/osiidoc/pages/163854/Kernel+Structure#Task-States>

Tasks - Zustände



*Mastering the FreeRTOS Real-Time Kernel v1.1.0

Tasks – Zustände zusammengefasst

■ Ready-State

- Wird nach Initialisierung gesetzt. Unsere Tasks können damit direkt starten.
- Signalisiert, dass ein Task ausgeführt werden kann.

■ Running-State

- Auf einem Single-Core System kann nur ein Task gleichzeitig laufen.
 - Nur ein Task kann im Running-State sein!
- Wir können in den Ready- oder Blocking State wechseln
 - Durch Kernelaufrufe: Für Delay- oder auch Ressourcenanfragen

■ Blocking-State

- Tasks in diesem Zustand bleiben blockiert, bis die angefragte Ressource frei wird...
- ... oder das zu erwartende Event eintritt (z.B. Delay-Time abgelaufen).
 - Wenn das passiert: Task wird automatisch in den Ready- oder sogar Running-State (wenn der Task die höchste Priorität hat) gesetzt.
- Zwingend benötigt, damit auch Tasks mit niedrigerer Priorität Ausführungszeit bekommen

Task – Management & API

- Wir müssen an einer zentralen Stelle unsere Tasks verwalten!
 - Z.B. als Array oder verkettete Liste
- Scheduler haben oft eine Task-Ready Liste, die alle Tasks enthält, die im „Ready“ State sind.
 - Macht Sinn, wenn wir dieselbe Task-Priorität mehrmals zulassen
- Wir benötigen eine API für unsere Task Informationen für:
 - Task Erstellung (TaskCreate)
 - Task Entfernung (TaskDelete)
 - Task ID und TCB zurückgeben (GetTaskId & GetTaskTCB)
 - Task Zustand zurückgeben und ändern (GetTaskState & SetTaskState)
 - Task Priorität zurückgeben (GetTaskPrio)

Kernel-Start – Die Initialisierung unseres Betriebssystems

- Während des Systemstarts werden wir
 - alle Tasks initialisieren, also den initialen Zustand für das Task-Objekt setzen.
 - den Speicher für unsere Tasks statisch allokalieren
 - die Tasks starten.
- Ein IDLE-Task muss erstellt werden
- Einige Betriebssysteme starten weitere Tasks wie Tasks zum Logging, Debugging oder Exception-Handling
- Abschließend wird der Scheduler aufgerufen, welcher den ersten Task in unserem System startet.

Kernel-Start – Die Initialisierung unseres Betriebssystems

1. Tasks initialisieren

1. Speicher allokieren
2. Task-Objekt initialisieren
3. Tasks in den Ready-State setzen

2. IDLE-Task erstellen

3. Scheduler aufrufen, welcher einen Task startet (abhängig von Faktoren wie Task-Priorität etc.)

Praktischer Teil

- Entwicklung des Task Managements
 - Was für Operationen brauchen wir für die Tasks?
 - Was für Informationen müssen wir speichern...
 - ... und welche Datenstrukturen brauchen wir dafür?
- Entwicklung des Dispatching
 - Wie wird das Dispatching gestartet?
 - Was passiert beim Dispatching? (Letzte Woche)
 - Wie muss das Dispatching umgesetzt werden?
 - Programming Manual Cortex-M4 lesen!
Exception Model, Instruction Set, Programmers model)
- Entwicklung des Schedulers:
 - Einfaches Round Robin: Feste Zeitscheiben, nach welchen unterbrochen wird.
 - (Erst einmal) Keine Priorität
 - Fester Ablaufplan (T1, T2, T3 → T1, T2, T3 → T1, T2, T3)

*Entwicklung des
Minimalsystems*

Zeitplan - Beispiel

Projektplan

